

Academic Kechafa Deployment Guide

This guide was written after scanning the current repository. It is tailored to this project, not a generic Flask template.

1. What I Found In Your Project

Current stack in this repo:

- Flask 3 with an app factory in kechafa_app/__init__.py
- Session-based authentication in routes/auth.py
- SQLite database logic hard-wired in database.py
- A partially prepared SQLAlchemy model layer in models.py
- File uploads stored locally under static/uploads/
- AI integration controlled by environment variables in kechafa_app/config.py
- Docker files present, but production still starts with python app.py

Important reality:

- You can deploy this project quickly on Render for a demo.
- You cannot get a truly professional, scalable deployment with the current database and media setup until you move away from SQLite and local file storage.

2. Biggest Risks To Fix Before Production

These are the main issues I found during the scan:

1. database.py uses Python sqlite3 directly, so DATABASE_URL alone will not switch the app to PostgreSQL.
2. Uploaded files are saved in static/uploads/, which is not the right long-term place for production video and book storage.
3. The current Dockerfile starts the development server with python app.py instead of Gunicorn.
4. A real .env file exists in the repo. Any real API keys or secrets in git should be rotated immediately and kept out of version control.
5. The repo contains venv/ and backup virtual environment files. Those should never be pushed to GitHub for deployment.

3. Recommended Production Architecture

Use this target architecture for a real-world app:

```
Users
|
v
Custom Domain + HTTPS
|
v
Render Web Service
|
+--> Gunicorn
+--> Flask App
|
+--> PostgreSQL
+--> Cloudinary or AWS S3 for media
+--> OpenRouter / OpenAI for AI features
+--> Redis later for cache and background jobs
```

Simple explanation:

- Render runs your Flask app on the internet.
- Gunicorn serves Flask in production.
- PostgreSQL stores users, courses, messages, books, and progress.
- Cloudinary or S3 stores videos, PDFs, and images.
- HTTPS protects logins and sessions.
- A custom domain makes the app look professional.

4. Best Deployment Strategy For This Repo

I recommend a two-phase approach:

Phase A: Deploy Fast

Use Render with the current Flask app, Gunicorn, and either:

- temporary SQLite on a persistent disk for testing and demos, or
- PostgreSQL only after you complete the database refactor

Phase B: Deploy Professionally

Upgrade the app to:

- PostgreSQL
- cloud media storage
- cleaned requirements
- better secret management
- background jobs for heavy tasks

This is the right path if you want a professional AI-powered e-learning platform.

5. Clean The Repo Before Deployment

Before pushing to GitHub, clean these items:

1. Make sure .env is ignored and remove any real secrets from git history if they were committed.
2. Do not push venv/ or venv_broken_backup/.
3. Do not rely on kechafa.db inside the repo for production.
4. Do not rely on static/uploads/ for long-term media.
5. Keep only source code, templates, static UI assets, docs, and tests in git.

Useful commands:

```
git status
git rm -r --cached venv venv_broken_backup
git rm --cached .env
```

If secrets were ever committed:

Rotate them first, then clean git history if needed.

6. Production-Friendly Project Structure

Keep your project like this:

```
ELearning/
|- app.py
|- wsgi.py
|- requirements.txt
|- .env.example
```

```
| - render.yaml                  # optional but recommended
| - Dockerfile
| - kechafa_app/
| - routes/
| - templates/
| - static/
| - docs/
| - tests/
```

Add a wsgi.py file for Gunicorn:

```
from kechafa_app import create_app
```

```
app = create_app("production")
```

Why this helps:

- app.py stays for local development.
- wsgi.py becomes the stable production entrypoint.
- Render and other hosts can run gunicorn wsgi:app.

7. How To Create A Good requirements.txt

Your current requirements.txt is close, but it mixes production and dev notes. For deployment, keep a real production file and optionally a separate dev file.

Recommended production requirements example

```
Flask==3.0.3
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Flask-Babel==4.0.0
Werkzeug==3.0.3
SQLAlchemy==2.0.31
openai==1.51.2
reportlab==4.2.2
arabic-reshaper==3.0.0
python-bidi==0.4.2
gunicorn==22.0.0
psycpg2-binary==2.9.9
Flask-Cors==5.0.0
cloudinary==1.40.0
```

Optional dev requirements

```
pytest==8.3.3
pytest-cov==5.0.0
flake8==7.1.1
Flask-Migrate==4.0.7
```

If you want to regenerate requirements

```
pip freeze > requirements.txt
```

For beginners, I recommend manually curating the file instead of blindly freezing everything. This keeps deployment clean and avoids shipping local-only packages.

8. Environment Variables

Your app already reads environment variables from kechafa_app/config.py. That is good.

For production, use environment variables in the hosting dashboard, not a real .env committed to git.

Recommended production variables

```
FLASK_ENV=production
SECRET_KEY=your-long-random-secret
PREFERRED_URL_SCHEME=https
FORCE_HTTPS=true
TRUST_PROXY_HEADERS=true
DEFAULT_LANGUAGE=fr
MAX_CONTENT_LENGTH_MB=250
```

```
ENABLE_REAL_AI=false
OPENROUTER_API_KEY=
OPENROUTER_FALLBACK_API_KEY=
OPENROUTER_MODEL=openrouter/free
OPENROUTER_MAX_TOKENS=512
OPENROUTER_APP_URL=https://your-domain.com
OPENROUTER_APP_TITLE=Academic Kechafa
```

```
DATABASE_URL=postgresql://user:password@host:5432/dbname
KECHAFA_DB=/var/data/kechafa.db
```

Notes:

- Use DATABASE_URL only after the database layer is upgraded for PostgreSQL.
- If you stay on SQLite temporarily on Render, set KECHAFA_DB to the mounted disk path.
- Never store real secrets in the repo.

9. Use Gunicorn Instead Of Flask Run

Do not use:

```
python app.py
```

Use:

```
gunicorn wsgi:app
```

Recommended start command:

```
gunicorn wsgi:app --workers 2 --threads 4 --timeout 120
```

Why Gunicorn is better:

- It is designed for production.
- It handles multiple workers.
- It is more stable behind Render or Railway.
- It avoids using Flask's development server on the public internet.

10. Step-By-Step Render Deployment

This is the easiest path for this project.

Step 1: Push the project to GitHub

```
git add .
git commit -m "Prepare app for deployment"
git push origin main
```

Step 2: Create a Render account

Go to:

<https://render.com/>

Step 3: Create a new Web Service

In Render:

1. Click New +
2. Choose Web Service
3. Connect your GitHub account
4. Select this repository
5. Choose the branch to deploy

Step 4: Set runtime information

Typical settings:

- Environment: Python
- Build Command: `pip install -r requirements.txt`
- Start Command: `gunicorn wsgi:app --workers 2 --threads 4 --timeout 120`

Step 5: Add environment variables

Add these in the Render dashboard:

```
FLASK_ENV=production
SECRET_KEY=your-secret-key
PREFERRED_URL_SCHEME=https
FORCE_HTTPS=true
TRUST_PROXY_HEADERS=true
MAX_CONTENT_LENGTH_MB=250
ENABLE_REAL_AI=false
```

If using SQLite temporarily:

```
KECHAFA_DB=/var/data/kechafa.db
```

If using PostgreSQL after migration:

```
DATABASE_URL=postgresql://...
```

Step 6: Deploy

Click deploy and watch the logs.

Step 7: Test the public URL

Test these routes:

- /
- /login
- /register
- /courses
- /library
- /messages
- /ai-coach

11. Quick Demo Deployment With SQLite

If your goal is to get online fast before the PostgreSQL refactor, do this:

1. Add `wsgi.py`
2. Add `gunicorn` to `requirements.txt`
3. Deploy on Render
4. Attach a persistent disk
5. Set `KECHAFA_DB` to the persistent disk path

Example:

KECHAFA_DB=/var/data/kechafa.db

This is acceptable for:

- demos
- portfolio projects
- small private testing

This is not ideal for:

- high traffic
- multi-instance scaling
- heavy concurrent writes
- long-term production

12. Professional Database Upgrade: SQLite To PostgreSQL

This is the most important production upgrade for your app.

Why SQLite is not enough

SQLite is good for local development and small demos, but not ideal for a real e-learning platform because:

- writes can become a bottleneck
- scaling is limited
- backups and recovery are weaker
- multi-instance deployment is difficult

Important repo-specific warning

Your config already defines DATABASE_URL, but the running app still uses database.py with sqlite3. That means this will not work yet:

```
DATABASE_URL=postgresql://...
```

You need a real migration step first.

Best migration path for this repo

The repo already contains models.py with Flask-SQLAlchemy. That is the cleanest bridge to PostgreSQL.

Recommended plan:

1. Wire models.py into app startup
2. Add Flask-Migrate
3. Move table creation away from raw sqlite3 startup
4. Migrate repositories and services from raw SQL helpers to SQLAlchemy or PostgreSQL-safe queries
5. Create a managed PostgreSQL database on Render
6. Set DATABASE_URL in production

Example app initialization target

```
from kechafa_app import create_app
from models import init_models, db
```

```
app = create_app("production")
init_models(app)
```

```
with app.app_context():
    db.create_all()
```

Add Flask-Migrate

```
pip install Flask-Migrate
```

Example:

```
from flask_migrate import Migrate
from models import db
```

```
migrate = Migrate()
```

```
def init_extensions(app):
    db.init_app(app)
    migrate.init_app(app, db)
```

Then:

```
flask db init
flask db migrate -m "initial postgres schema"
flask db upgrade
```

Create PostgreSQL on Render

Create a managed PostgreSQL instance in Render, copy its internal or external connection string, and save it as:

```
DATABASE_URL=postgresql://...
```

Final goal

After the refactor, your production app should use PostgreSQL for:

- users
- courses
- books
- lessons
- enrollments
- messages
- notifications
- AI conversation history

13. How To Connect Flask To PostgreSQL In Production

If you complete the SQLAlchemy migration, use:

```
app.config["SQLALCHEMY_DATABASE_URI"] = os.environ["DATABASE_URL"]
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
```

If you keep a custom repository layer, make sure every connection reads from DATABASE_URL. Do not leave production logic hard-coded to:

```
sqlite3.connect("kechafa.db")
```

14. Media Handling For Videos And PDF Books

This is your second major production upgrade.

Right now, files are stored in:

- static/uploads/books/
- static/uploads/book-covers/
- static/uploads/content/

That is fine locally, but not the best choice on cloud hosting.

Best option for your app

Use:

- Cloudinary for images and videos
- AWS S3 for PDFs, books, and larger file storage

Easy rule

- Videos: Cloudinary
- PDFs and documents: AWS S3 or Cloudinary raw files
- Small images: Cloudinary

Cloudinary example in Flask

```
import cloudinary
import cloudinary.uploader

cloudinary.config(
    cloud_name=os.environ["CLOUDINARY_CLOUD_NAME"],
    api_key=os.environ["CLOUDINARY_API_KEY"],
    api_secret=os.environ["CLOUDINARY_API_SECRET"],
    secure=True,
)

result = cloudinary.uploader.upload(
    file,
    resource_type="video",
    folder="academic-kechafa/lessons"
)

video_url = result["secure_url"]
```

Then save video_url in the database instead of a local file path.

AWS S3 example in Flask

```
import boto3
from uuid import uuid4

s3 = boto3.client(
    "s3",
    aws_access_key_id=os.environ["AWS_ACCESS_KEY_ID"],
    aws_secret_access_key=os.environ["AWS_SECRET_ACCESS_KEY"],
    region_name=os.environ["AWS_REGION"],
)

filename = f"books/{uuid4()}.pdf"
s3.upload_fileobj(file, os.environ["S3_BUCKET_NAME"], filename)

file_url =
f"https://{os.environ['S3_BUCKET_NAME']}.s3.amazonaws.com/{filename}"
```

Recommended database storage pattern

Store only metadata in PostgreSQL:

- title
- original filename
- provider name
- provider public id
- secure URL
- uploaded by

- created at

Do not store large binary files in the database.

15. Frontend Integration: HTML Or React

You currently have server-rendered HTML templates. That is the easiest deployment model.

Option A: Keep Flask templates

This is simplest and fastest:

- Flask serves HTML
- Flask handles login
- Flask reads and writes data
- same domain, simpler cookies, simpler deployment

Option B: Add a React frontend later

If you separate the frontend and backend:

- React frontend can live on Vercel, Netlify, or Render
- Flask API can stay on Render
- the two apps communicate through JSON APIs

Example React fetch call

```
const response = await fetch("https://api.yourdomain.com/api/courses", {
  method: "GET",
  credentials: "include",
  headers: {
    "Content-Type": "application/json"
  }
});
```

16. How To Handle CORS

You do not currently have Flask-CORS configured. That is okay while you use server-rendered templates on the same domain.

If you split React and Flask into separate origins, add Flask-Cors.

Install:

```
pip install Flask-Cors
```

Example:

```
from flask_cors import CORS

def create_app(config_name=None, config_overrides=None):
    app = Flask(__name__, template_folder="../templates",
                static_folder="../static")

    CORS(
        app,
        resources={r"/api/*": {"origins": ["https://app.yourdomain.com"]}},
        supports_credentials=True,
    )

    return app
```

Important:

- If you use cookies across domains, configure SameSite, Secure, and CSRF protection carefully.

- For beginners, keeping Flask templates on the same domain is easier than splitting React and Flask immediately.

17. Security Checklist

Your app already has login and register pages. That is a good start.

To harden security for production:

1. Set a strong SECRET_KEY
2. Keep FLASK_ENV=production
3. Enable FORCE_HTTPS=true
4. Enable TRUST_PROXY_HEADERS=true
5. Keep SESSION_COOKIE_SECURE=True in production
6. Rotate any leaked API keys
7. Add rate limiting for login and AI endpoints
8. Add CSRF protection for forms
9. Add email verification and password reset
10. Restrict admin routes carefully

Protecting API routes

For HTML views, you already use decorators like:

- login_required
- admin_required
- instructor_required

For future JSON APIs, keep the same access rules and return JSON errors.

Example protected API route

```
@app.route("/api/me")
@login_required
def me():
    return {"ok": True}
```

18. Custom Domain And HTTPS

After deployment works on the generated Render URL:

1. Buy a domain from a registrar
2. Add the domain in the Render dashboard
3. Update DNS records at your domain provider
4. Wait for DNS propagation
5. Test HTTPS

For this project, also set:

```
PREFERRED_URL_SCHEME=https
FORCE_HTTPS=true
TRUST_PROXY_HEADERS=true
```

Why these matter:

- PREFERRED_URL_SCHEME=https makes generated links use HTTPS
- FORCE_HTTPS=true redirects insecure traffic
- TRUST_PROXY_HEADERS=true prevents redirect loops behind a proxy

19. Common Deployment Errors And Fixes

Error: ModuleNotFoundError: No module named gunicorn

Fix:

Add gunicorn to requirements.txt and redeploy.

Error: app starts locally but crashes on Render

Common reasons:

- missing environment variables
- wrong start command
- code still assuming local paths
- .env values not copied into the hosting dashboard

Error: sqlite3.OperationalError: unable to open database file

Fix:

- use a valid writable path
- on Render, point KECHAFA_DB to the persistent disk mount

Error: uploaded files disappear after redeploy

Cause:

- files were stored on local service disk

Fix:

- move media to Cloudinary or S3

Error: redirect loop with HTTPS

Fix:

```
TRUST_PROXY_HEADERS=true  
FORCE_HTTPS=true  
PREFERRED_URL_SCHEME=https
```

Error: large uploads fail

Fix:

- increase MAX_CONTENT_LENGTH_MB
- use direct-to-cloud uploads for large videos

Error: PostgreSQL URL is set but app still uses SQLite

Cause:

- current repo still reads from database.py using sqlite3

Fix:

- finish the database migration before relying on PostgreSQL in production

20. Scaling Tips

When traffic grows, do this:

1. Move media out of the app server
2. Use PostgreSQL instead of SQLite
3. Add Redis for caching and queues
4. Move slow jobs to background workers
5. Paginate feeds, books, and messages
6. Compress images and thumbnails
7. Use CDN-backed media URLs

8. Add indexes to frequently queried tables
9. Monitor logs, response time, and database load
10. Split API, worker, and frontend later if needed

21. Basic Performance Tips

For this e-learning platform:

- lazy-load videos and book covers
- generate thumbnails instead of loading original media everywhere
- cache category lists and dashboard counters
- avoid re-running heavy queries on every request
- move AI calls to background jobs if they become slow
- serve static assets with cache headers

22. Suggested Real-World Upgrade Roadmap

Stage 1: Launchable

- add wsgi.py
- use Gunicorn
- deploy on Render
- use SQLite only for demo if needed

Stage 2: Professional backend

- finish SQLAlchemy integration using models.py
- add Flask-Migrate
- move to PostgreSQL
- clean repository and secrets

Stage 3: Professional media pipeline

- Cloudinary for video and images
- S3 for books and large documents
- direct browser upload for big files

Stage 4: Product quality

- password reset
- email verification
- audit logs
- analytics dashboard
- admin moderation tools
- background workers

23. Bonus Ideas For An AI-Powered E-Learning Platform

These are strong next features for your product:

1. AI quiz generation from lessons and books
2. AI summaries for PDFs and course videos
3. Semantic search across books, lessons, and conversations
4. Personalized learning paths
5. Progress dashboards for students and instructors
6. Certificates with verification pages

7. Video transcription and subtitles
8. Study reminders by email or push notification
9. Role-based admin analytics
10. Recommendation engine for courses and books

24. My Recommended Exact Path For You

If you want the smartest path with the least wasted time, do this in order:

1. Clean the repo and rotate any committed secrets
2. Add wsgi.py
3. Add gunicorn to production requirements
4. Deploy to Render with SQLite on a persistent disk for a first online version
5. Confirm the public app works
6. Finish the PostgreSQL migration using the existing models.py foundation
7. Move videos and books to Cloudinary or S3
8. Redeploy with PostgreSQL and cloud media
9. Add a custom domain
10. Turn on live AI only after the app is stable

25. Final Summary

You already have a good Flask foundation:

- authentication
- courses
- books
- messages
- notifications
- AI chat

The project is close to being deployable, but a professional production version needs four major upgrades:

1. Gunicorn instead of Flask dev server
2. PostgreSQL instead of SQLite
3. Cloud media storage instead of local static/uploads
4. Stronger secret and deployment hygiene

If you follow this guide, you can get:

- a fast public demo first
- then a production-ready architecture for a real application

26. Useful Links

Official home pages and docs entry points:

- Render: <https://render.com/>
- Render docs: <https://render.com/docs>
- Railway: <https://railway.com/>
- Railway docs: <https://docs.railway.com/>
- Cloudinary docs: <https://cloudinary.com/documentation>
- AWS S3 docs: <https://docs.aws.amazon.com/s3/>
- Flask deployment docs: <https://flask.palletsprojects.com/>

- Unicorn docs: <https://docs.gunicorn.org/>